Test Plan and Report

Vitae | 2026-03-10

Sprint 1 System Test Scenarios

1. Sign In (2 points): As a user, I can sign in so that I can access my data

Acceptance Criteria	**System Test Scenario**

Scenario 1: Create an account
- Given that a user doesn't have an account
- When they Sign Up
- Then they should have an account and can access all the pages
 	
Scenario 2: Sign in to an account
- Given that a user has an account
- When they Sign In
- Then they should be logged in to their account
	Scenario 1: Create account (**PASSES**)
	- Navigate to `/` (root page)
	  - Click on 'Sign Up' button
	  - Click 'Continue with Google'
	  - Select a Google account
	  - Get rerouted to `/home` automatically and view the home page
	- The user should have their account set up
	
	Scenario 2: Sign in to an account (**PASSES**)
	- Navigate to `/` (root page) while logged out
	  - Click on 'Sign In' button
	  - Click 'Continue with Google'
	  - Select a Google account
	  - Get rerouted to `/home` automatically and view the home page
	- The user should be able to log in to the same account they originally created

Unit Tests: `./__tests__/auth/`

---

2. Session Persistence (2 points): As a user, I want to stay logged in across browser sessions so that I don't have to sign in every time.

Acceptance Criteria	**System Test Scenario**

Scenario 1: Stay logged in after refresh
- Given that a user is signed in
- When they refresh the page or close and open it again
- Then they should still be signed in
	Scenario 1: Stay signed in (**PASSES**)
	- Navigate to any page.
	  - Click refresh or close the tab
	  - Reopen the tab if closed
	- The user should still be logged in

Unit Tests: `./__tests__/auth/`

---

3. Add Canon Items (6 points): As a job applicant, I need to be able to enter my entire career history so that I can draw from it when making resumes.

Acceptance Criteria	**System Test Scenario**

Scenario 1: Successfully add a canon item
- Given that a user is on the career history page
- When they fill out the “add item” form with valid details and select an item type
- Then the item should be saved and appear in their canon list, grouped by type
	Scenario 1: Add a canon item (**PASSES**)
	- Navigate to /home
	  - Select an item type
	  - Verify the form field is updated to match the selected type
	  - Fill in all required fields with valid data
	  - Submit the form
	  - The item should appear in the canon list with the correct type group
	  - The item should persist via POST /canon and be retrievable upon page refresh

Scenario 2: Dynamic form fields based on item type
- Given that a user is filling out the “add item” form
- When they select a different item from the type selector
- Then the form fields should update dynamically to match the selected type
	Scenario 2: Update Canon form (**PASSES**)
	- Navigate to /home
	  - Select an item type
	  - Verify the form field is updated to match the selected type
	  - Select a different item type
	  - Verify the form field is updated to match the new selected type

Scenario 3: Handle form errors
- Given that a user submits the “add item” form with missing or invalid data
- When the form is submitted
- Then an error message should be displayed and
	Scenario 3: Handle errors on add (**PASSES**)
	- Navigate to /home
	  - Add item
	  - Submit the form with the required fields empty
	  - An error message should be displayed
	  - No new item should be added to the list

Unit Tests: `./__tests__/canon/add.test.tsx`

---

4. Edit Canon Items (4 points): As a job applicant, I need to be able to edit my career history so that I can correct errors.

Acceptance Criteria	**System Test Scenario**
Scenario 1: Successfully edit a canon item - Given that a user has existing career history items	Scenario 1: Edit a canon item (**PASSES**) - Navigate to /home - Click “Edit” on the Canon item

- When they edit an item and submit the “edit item” form with updated details - Then the item should be updated and reflect the new information in the canon list	- Modify one or more fields in the form - Click Update - The updated item should appear in the canon list with new values - The update should be persisted via PUT /cannon/:item-id and retrievable upon refresh

Unit Tests: `./__tests__/canon/edit.test.tsx`

---

5. Deleting Canon Items (2 points): As a job applicant, I can delete career history items so that I can keep my records accurate.

Acceptance Criteria	**System Test Scenario**

Scenario 1: Successfully delete a Canon item
- Given that a user has existing career history items
- When they select an item and confirm deletion
- Then the item should be permanently removed from their canon list
	Scenario 1: Delete a canon item (**PASSES**)
	- Navigate to /home
	  - Select an existing item and click “Delete”
	  - Confirm the deletion in the form
	  - The item should no longer appear in the canon list
	  - The deletion should be persisted via DELETE /canon/:item-id, and the item should not appear on refresh

Scenario 2: Cancel Deletion
- Given that a user initiates a delete action
- When they cancel the deletion
- Then the item should remain unchanged in their canon list
	Scenario 2: Cancel Deletion (**PASSES**)
	- Navigate to /home
	  - Select an existing item and click “Delete”
	  - Cancel the deletion prompt
	  - The item should be visible and unchanged in the canon list

Unit Tests: `./__tests__/canon/add.test.tsx`

---

6. Viewing Canon Items (4 points): As a job applicant, I need to save and view my entire career history so that I can refer back to it at any time.

Acceptance Criteria	**System Test Scenario**
Scenario 1: View Canon items grouped by type - Given that a user has career history items saved - When they are on the Career History Page - Then they should see their items displayed and grouped by type	Scenario 1: View Canon items (**PASSES**) - Navigate to the career history page - Items should be viewable and grouped by their type (i.e., work experience, education, skills) - Data should be fetched from the GET /canon API endpoint and match the user's stored records
Scenario 2: View Canon with no items - Given that the user has no career history items saved yet - When they are on the Career History Page - Then they should see an empty state message that says no items have been added yet	Scenario 2: View Canon with no items (**PASSES**) - The page should load without errors - Page should state there are no Canon items yet - No item groups or content should be displayed

Unit Tests: `./__tests__/canon/view.test.tsx`

Sprint 2 System Test Scenarios

1. Select Items for Resume (2 points): As a job applicant, I can select which career history items appear in my current resume so that I can tailor content for a specific job opportunity.

Acceptance Criteria	**System Test Scenario**

Scenario 1: Select and deselect resume items
- Given that a user is on the Resume Builder page
- When they check or uncheck a checkbox next to a career history item
- Then that item should appear or disappear from the resume preview, and the selection should be saved automatically
 	
Scenario 2: Selection persists after syncing
- Given that a user has selected items on the Resume Builder page and clicked the Save button
- When they navigate away and come back
- Then their previously selected items should still be checked
	Scenario 1: Select an item (**PASSES**)
	- Navigate to \resume
	  - Check or uncheck a checkbox next to any career history item
	  - The item should appear or disappear in the resume preview on the right panel based on the selection.
	
	Scenario 2: Selection persists (**PASSES**)
	- Navigate to /resume
	  - Select an item in /resume
	  - Click the save resume button (can cancel naming or proceed)
	  - Navigate to /home and come back to /resume
	  - The same item should be selected

Unit Tests: `N/A`

---

2. Resume Reordering (3 points): As a user, I can reorder sections and items within my resume so that I can prioritize what matters most for an application or audience.

Acceptance Criteria	**System Test Scenario**

Scenario 1: Reorder sections within a resume
- Given that a user has multiple sections in their resume
- When they drag and drop a section to a new position
- Then the sections should reflect the new order, and the changes should be saved
	Scenario 1: Reorder Sections(**PASSES**)
	- Navigate to /resume
	  - Drag one section above or below another one
	  - The sections should visually reorder immediately
	  - The new section order should be auto-saved to the working state
	  - On page refresh, the updated section order should persist

Scenario 2: Reorder item within a section
- Given that a user has multiple items within a resume section
- When they drag and drop an item to a new position within that section
- Then the items should reflect the new order, and the change should be saved
	Scenario 2: Reorder Sections(**PASSES**)
	- Navigate to /resume
	  - Drag one item above or below another one within the same section
	  - The items should visually reorder immediately
	  - The new item order should be auto-saved to the working state
	  - On page refresh, the updated item order should persist

Scenario 3: Handle Edge cases in ordering
- Given that a user has a section with no items
- When they view or attempt to reorder that section
- Then the UI should manage it gracefully without errors or broken behavior
	Scenario 3: Handle Edge Cases(**PASSES**)
	- Navigate to /resume
	  - Open a section with only one item - drag and drop should work properly
	  - Attempt any reorder action and confirm no errors occur

Unit Tests: `N/A`

---

3. Save Named Resumes (4 points): As a user, I can name and save my current resume so that I can retrieve it later.

Acceptance Criteria	**System Test Scenario**

Scenario 1: Save Resume Button Functions
- Given that a user has a resume
- When they click on the save resume button
- Then they should see a modal pop up, asking them to fill out the required fields
Scenario 2: Label a resume and save it
	Scenario 1: Label and save a resume (**PASSES**)
	- Navigate to `/resume`
	  - Click on “Save Resume”
	  - A modal/form should pop up asking the user to fill in `Save To`, `Resume Name`, and optionally a `Version

- Given that a user has a resume and has filled out the required fields - When they submit the form and navigate to their version history - Then they should see their newly saved resume version appear as a card with the relevant details Scenario 3: Restore a resume and save it again - Given that a user has an existing resume version - When they restore this version, modify their resume, and save their resume again - Then they should see a newly saved resume version appear as a card above the previous version.	Note.` - The user should see the modal with the appropriate fields  Scenario 2: Label and save a resume (**PASSES**) - Navigate to `/resume`   - Click on “Save Resume”   - A modal/form should pop up   - Fill in `Resume Name` as `Full-Stack SWE`   - Press `Save`   - Navigate to `/versions` - The user should see a new card component containing details of the date and time they saved and the resume name as `Full-Stack SWE`  Scenario 3: Restore a resume and save it again (**PASSES**) - Navigate to `/versions` - Click on any existing version and click `restore`, for example `Full-Stack SWE`   - Click `Restore`   - Wait to be automatically navigated to `/resume?[url params]` - Modify the resume in any way through the resume-builder section - Click on “Save Resume”   - A modal/form should pop up   - Fill in `Resume Name` as `Backend SWE 2`   - Press `Save`   - Navigate to `/versions` - Then they should see a newly saved resume version appear as a card above the previous version with the name `Backend SWE 2` as well as the time it was saved.

Unit Tests: `./__tests__/versions/SaveResumeButton.test.txt`

---

4. View Named Resumes (2 points): As a user, I can view my saved resumes so that I can look at past versions.

Acceptance Criteria	**System Test Scenario**
Scenario 1: View list of saved resumes - Given that a user has saved at least one resume - When they navigate to the Version History page - Then they should see a list of their saved resumes with name and date created. Scenario 2: Empty state when no resumes saved - Given that a user has no saved resumes - When they navigate to the Version History page - Then they should see an empty state message indicating that no resumes have been saved yet.	Scenario 1: View saved resumes (**PASSES**) - Navigate to /versions - The page should display a list of saved resumes - Each resume should show its name and the date it was created Scenario 2: Empty state (**PASSES**l) - Navigate to /versions with no saved resumes - An empty state message should be visible instead of a list.

Unit Tests: `N/A`

---

5. Restore Named Resume (3 points): As a user, I can restore a saved resume to my working state so that I can continue editing or reuse it.

Acceptance Criteria	**System Test Scenario**

Scenario 1: Restore a resume version and view it as expected in the resume builder tab
- Given that a user has a saved resume version
- When they restore their resume
- Then they should be able to view that resume and edit it
	Scenario 1: Restore a resume version (**PASSES**)
	- Navigate to `/versions`
	  - Click on “Restore” on a resume version card
	  - A modal/form should pop up
	  - Click on `Restore`
	  - Get rerouted automatically to `/resume`
	- The user should see that their resume is filled with the appropriate values according to what was in that resume version

Unit Tests: `N/A`

---

6. Live preview (6 points): As a user, I can see a live preview of my resume as a formatted PDF so that I know what the final output looks like.

Acceptance Criteria	**System Test Scenario**
Scenario 1: Show loading state on initial preview - Given a user is on the Resume Builder page - When the live preview is compiling for the first time - Then they should see a loading message indicating the preview is being generated.	Scenario 1: Initial preview loads (**PASSES**) - Navigate to /resume - Observe the preview pane immediately after page load - The UI should display “Generating preview…”
Scenario 2: Render initial preview successfully - Given a user is on the Resume Builder page - When the initial preview compilation succeeds - Then they should see a formatted preview of their resume.	Scenario 2: Render initial preview successfully (**PASSES**) - Navigate to /resume with a successful preview compile response - Wait for initial compile to complete - Verify a formatted preview appears and “Generating preview…” is no longer visible
Scenario 3: Update preview after resume edits - Given a user already sees a resume preview - When they edit their resume content - Then the preview should automatically update to reflect the latest changes.	Scenario 3: Update preview after resume edits (**PASSES**) - Navigate to /resume and wait for initial preview render - Edit resume content - Wait for preview refresh - Verify that preview automatically changes to reflect the latest edit
Scenario 4: Handle initial preview failure - Given a user is on the Resume Builder page - When the initial preview compilation fails - Then they should see a blocking preview error message with details.	Scenario 4: Handle initial preview failure (**PASSES**) - Navigate to /resume with initial compile forced to fail - Verify the preview area shows “Preview failed” - Verify the compile error details are visible to the user
Scenario 5: Handle update failure without losing current preview - Given the user already has a preview visible - When a later preview update fails after an edit - Then the last successful preview should remain visible and a non-blocking update error should be shown.	Scenario 5: Handle update failure without losing current preview (**PASSES**) - Navigate to /resume and wait for successful preview render - Make an edit and force the next preview update request to fail - Verify previously rendered preview is still visible - Verify a non-blocking banner saying “Preview update failed” is showing with details
Scenario 6: Recover from update failure - Given a user already has a preview visible and an update to the preview failed	Scenario 6: Recover from update failure (**PASSES**) - Start from Scenario 5 state (non-blocking error update visible

- When a subsequent preview update succeeds - The error message should clear and the preview should show the newest content. Scenario 7: Ignore stale out-of-order responses - Given a user makes multiple quick edits that trigger multiple preview requests - When an older request finishes after a newer request - Then the user should only see the newest preview result.	- Make another edit and return a successful compile response - Verify “Preview update failed” banner disappears - Verify preview shows newest content Scenario 7: Ignore stale out-of-order responses (**PASSES**) - Navigate to /resume - Trigger two fast consecutive edits (Request A, then Request B) - Return Request B first, then return delayed Request A - Verify preview shows Request B result - Verify preview does not revert to Request A content after A finally returns

Unit Tests: `__tests__/typst/preview.test.tsx`

Sprint 3 System Test Scenarios

1. Override (3 points): As a user, I can override a canon or resume item’s content for a specific resume without changing the original so that I can tailor it for different audiences. Berat Ercevik

Acceptance Criteria	**System Test Scenario**

Scenario 1: Edit any item’s properties in the resume builder without changing its original version
- Given that a user has a resume
- When they edit their resume in the resume builder page
- Then they should be able to see those edits only be reflected in that version and not change their canon items
	Scenario 1: Edit resume item without changing original (**PASSES**)
	- Navigate to `/resume`
	  - Click on the pencil icon in the top right of any resume item card
	  - An edit modal should pop up, almost exactly the same as in `/home`
	  - Change at least one field
	  - Click `Save Override`
	  - The pencil icon should be highlighted, indicating that the current item is different than canon
	  - Navigate to `/home`
	- The user should see that the resume item they changed in `/resume` has not been changed in `/home` (their canon)

Unit Tests: `./__tests__/resume-builder/edit-override.test.tsx`

---

2. Restore (4 points): As a user, I can restore a previous version to my working state so that I can reiterate on past resumes. Alex Lio

Acceptance Criteria	**System Test Scenario**

Scenario 1: Restore a previous resume version to working state
- Given that a user has at least one previously saved resume version
- When they choose to restore that previous version
- Then the selected version should become their current working state
- And all resume items, overrides, and content from that version should appear in the resume builder
 	
Scenario 2: Restoring a previous version does not delete version history
- Given that a user has multiple saved resume versions
- When they restore one of the earlier versions
- Then the restored version should be loaded into the working state. Other versions should persist
	Scenario 1: Restore previous version to working state (**PASSES**)
	- Navigate to /versions
	  - Open the version history or saved resumes section
	  - Select a previously saved version that is different
	  - Click restore on that version
	  - Confirm the restore action
	  - Verify that the resume builder reloads with the content from the selected previous version
	- Check that the resume items, text, and any overrides match the restored version
	- Make a small edit to confirm the restored version is now the active working state
	
	Scenario 2: Restore does not remove saved history (**PASSES**)
	- Navigate to the saved versions page
	- Restore an older version
	- After restore, reopen the saved versions/history page
	- Verify that the other saved versions are still present
	- Verify that the restored version is still represented in history and has not deleted any other entries

Unit Tests: `./__tests__/versions/restore-with-archive.test.ts`

---

3. Visualize (4 points): As a user, I would like to be able to visualize my career history Omer Boztepe

Acceptance Criteria	**System Test Scenario**

Scenario 1: View version history in an alternative view (git-tree)
- Given that a user has saved at least one resume version.
- When they navigate to the Version
	Scenario 1: Switch to tree view (**PASSES**)
	- Navigate to /versions
	  - Click the toggle button on any resume group
	  - The git-tree graph should

History page.
- Then they should see a button to switch between list and tree view, and clicking it should display their versions as a git graph or as a list.
 	
Scenario 2: View version details by clicking a node.
- Given that a user is on the Version History Page in tree view.
- When they click on a node in the tree
- Then they should see the version name date and restore/delete buttons for that version.
	appear that shows dots for each version.
	
	Scenario 2: Click a node (**PASSES**)
	- Navigate to /versions in tree view
	  - Click on any dot/node in the tree
	  - A details card should appear that displays the version name and restore/delete buttons.

Unit Tests: `N/A`

---

4. Labelling Version History (1 point): As a user, I would like to be able to see which version I'm currently on and see it while previewing the resume. Berat Ercevik

Acceptance Criteria	**System Test Scenario**

Scenario 1: View the current resume version I'm editing
- Given that a user has a resume
- When they make a restore from a resume version
- Then they should see what version they are on in the resume builder page
	Scenario 1: View current resume version (**PASSES)**
	- Navigate to `/versions`
	  - Restore a resume by clicking on `Restore` then `Restore` again on the pop-up
	  - The user should be rerouted to `/resume` automatically
	- The user should see "Version: [versionName]" and "Updated at: [TIME]."

Unit Tests: `./__tests__/resume-builder/version-display.test.tsx`

---

5. PDF Export (2 points): As a user, I can download a PDF of my resume so that I can upload it to job sites. Njeri Gachoka

Acceptance Criteria	**System Test Scenario**
Scenario 1: Export PDF from resume page - Given a user is on /resume with resume content - When they click Export PDF - Then a PDF download should start with filename resume.pdf.	Scenario 1: Export PDF from resume page (**PASSES**) - Navigate to /resume - Click Export PDF - Verify a request is made to /api/resume/compile/pdf - Verify browser download starts as resume.pdf.

Scenario 2: Export in progress state - Given a user clicks Export PDF - When PDF generation is in progress - Then the export button should show loading state and prevent duplicate clicks. Scenario 3: Export failure handling - Given a user clicks Export PDF - When PDF generation fails - Then the user should be informed that export failed (and be able to retry).	Scenario 2: Export in progress state (**PASSES**) - Click Export PDF - Verify button enters loading state and is disabled - Verify button returns to normal after request completes. Scenario 3: Export failure handling (**PASSES**) - Force /api/resume/compile/pdf to return error - Click Export PDF - Verify failure feedback is shown and user can click Export PDF again.

Unit Tests: `./__tests__/resume-builder/pdf-export.test.tsx`

---

6. View Templates (4 points): As a user, I can view multiple preset templates to see which ones would best fit the role I'm applying to. Njeri Gachoka

Acceptance Criteria	**System Test Scenario**
Scenario 1: View Available preset templates - Given that a user is on the resume builder page - When they click the template button - Then they should see multiple preset templates displayed with a preview of each	Scenario 1: View preset template (**PASSES**) - Navigate to /resume. - Click the resume template's name above the resume - All template previews should appear with their name without errors

Unit Tests: `./__tests__/resume-builder/template-view.test.tsx`

---

7. Use Templates (2 points): As a user, I can use a template for my resume so that I can match the resume's visual style to different industries or preferences. Keith Tran

Acceptance Criteria	**System Test Scenario**

Scenario 1: Apply template to a resume
- Given that a user has clicked the resume template button
- When they click on the template, they like
- Then the resume should update to reflect the chosen template's visual style, and the
	Scenario 1: Apply a template (**PASSES**)
	- Navigate to /resume
	  - Click the resume template button
	  - Select a template
	  - The resume preview should be updated to reflect the selected

selection should be saved to the working state Scenario 2: Template persists with the resume - Given that a user has applied a template to their resume - When they return to the resume later - Then the selected template should still be applied	template's style - The chosen template should be saved to the working state and persist on page refresh Scenario 2: Template persists across sessions (**PASSES**) - Apply a template to a resume and save - Go to a different page/reopen the resume - The template should still be applied and visible in the preview

Unit Tests: `N/A`

Sprint 4 System Test Scenarios

1. Diff check (3 points): As a user, I can compare two versions of my resume side-by-side so that I can see what changed between resume submissions.

Acceptance Criteria	**System Test Scenario**
Scenario 1: View a side-by-side diff of two resume versions - Given that a user has at least two saved versions of their resume - When they select two versions to compare - Then the system should display both versions side-by-side with the differences visually highlighted (e.g., additions, deletions, or changes)	Scenario 1: Side-by-Side Resume Diff (**PASSES**) - Navigate to /versions - Ensure at least two saved versions of a resume exist (e.g., "Version 1" and "Version 2") - Open the diff/compare view and select the two versions to compare - The UI should display both versions side-by-side - Differences between the two versions (added, removed, or modified content) should be visually distinguishable - Unchanged content should appear the same in both panes

Unit Tests: `N/A`

2. Quick build (3 points): As a user, I can paste a job description and see which canon items are most relevant so that I can quickly build a targeted resume.

Acceptance Criteria	**System Test Scenario**

Scenario 1: Reorder and reselect resume items based on the Job Description
- Given that a user has a job description
- When they enter it and press “Run AI Prioritization.”
- Then their resume items should be reordered and reselected accordingly, and see the result in the resume preview pane.
	Scenario 1: Reorder and reselect Resume (**PASSES**)
	- Navigate to /resume
	  - Enter a job description into the Job Description text input box
	  - Click “Run AI Prioritization”
	  - The resume-builder should be reordered, and certain elements may be toggled on or off
	  - The resume-preview should reflect those changes

Unit Tests: `N/A`

3. AI tailoring (4 points): As a user, I can get AI-suggested revisions of my experiences for a specific role so that I can tailor my language efficiently.

Acceptance Criteria	**System Test Scenario**

Scenario 1: Generate AI bullet tweaks for a resume item
- Given that a user has a saved description in the AI Tailoring Studio
- When they switch to AI Tailor mode and click the sparkles button on any item
- Then a modal should appear showing the item and a Generate button
 	
Scenario 2: Review and apply AI suggestions
- Given that a user has generated AI suggestions in the AI Tailor Item modal
- When they review the word diff showing red/green changes
- Then they should be able to click Apply to save the changes or Discard to cancel
	Scenario 1: Generate tweaks (**PASSES**)
	- Navigate to /resume
	  - Set up the context in AI Tailoring Studio based on preferences (job description/audience).
	  - Click Save Context
	  - Click the sparkles button on any item.
	  - Click generate, and a word diff should appear showing suggested changes
	
	Scenario 2: Apply Tweaks (**PASSES**)
	- After generating suggestions in the modal
	- Click Apply
	- The content of the item should be changed based on AI
	- The item's bullets in the resume preview should reflect the AI suggestions

Unit Tests: `N/A`

4. View Tailoring Options (6 points): As a user, I want to see the space of tweaks that can reasonably be made to my resume item so that I don't have to reprompt an LLM over and over with ways to tweak it.

Acceptance Criteria	**System Test Scenario**
Scenario 1: View tailoring controls - Given a user is on /resume and selects AI Tailoring - When they expand AI Tailoring Studio - Then they should see all four tailoring sliders (Domain Focus, Role Framing, Technical Depth, Detail Level Scenario 2: View tweak range and descriptor - Given the AI Tailoring Studio is open	Scenario 1: View tailoring controls (**PASSES**) - Navigate to /resume - Select AI Tailor - Click AI Tailoring Studio to expand - Confirm that Domain Focus, Role Framing, Technical Depth, and Detail Level sliders are shown Scenario 2: View tweak range and descriptor (**PASSES**)

- When the user views or adjusts any slider - Then they should see the low/high ends and current descriptor for that slider so they understand what tweaks are being requested	- With AI Tailoring Studio expanded, inspect Domain Focus slider - Confirm low/high labels are shown (Generalist vs. Domain-specific) - Confirm descriptor updates to reflect direction/intensity (i.e. Strongly domain-specific)

Unit Tests: `./__tests__/tailor/tailoring-studio-view.test.tsx`

5. Archive Bin (4 points): As a user, I would like deleted career items to be recoverable from a "Recently Deleted" bin so I can restore them or view them in older resume snapshots.

Acceptance Criteria	**System Test Scenario**
Scenario 1: Delete Item from canon history - Given a saved canon item is in the list, it should show up in the bin instead of being permanently deleted Scenario 2: Restore from recently deleted - Given that a user has one in there - When item is clicked restore, it should show up again in canon items Scenario 3: Permanently Delete - Given user has an item in the recently deleted bin, user should be able to delete the item permanently Scenario 4: View deleted items in older resume snapshots - Given that a user has a saved resume version that references a career item which was later deleted - When they restore that older version in Version History - Then the deleted item should still appear in the restored resume snapshot and preview Scenario 5: Expire archived items after 30 days - - Given that an item has remained in the Recently Deleted bin for more than 30 days - When the archive is loaded or a	Scenario 1: Delete item to archive (**PASSES**) Navigate to /home - Select an existing career item and click Delete - Confirm the deletion - The item should disappear from the main canon list - Open the Recently Deleted bin the deleted item should appear there Scenario 2: Restore archived item (**PASSES**) Navigate to /home - Open the Recently Deleted bin - Click Restore on an archived item - The item should be removed from the Recently Deleted bin - The item should reappear in the active Career History list Scenario 3: Permanently delete archived item (**PASSES**) - Navigate to /home - Open the Recently Deleted bin - Click Delete Forever on an archived item - Confirm the deletion if prompted - The item should disappear from the Recently Deleted bin - The item should not return on page refresh Scenario 4: Restore older version with deleted item (**PASSES**) - Navigate to /versions

restore is attempted - Then the expired item should no longer appear in the Recently Deleted bin and should not be restorable	- Choose a saved resume version that includes an item later deleted from Career History - Click Restore on that version - Get redirected to /resume - The restored resume preview should still show the deleted item in that snapshot - The deleted item should not be re-added to the active Career History list unless explicitly restored from Recently Deleted Scenario 5: Archive expiry behavior (**PASSES**) - Navigate to /home - Open the Recently Deleted bin after an archived item has passed the 30-day window - The expired item should no longer appear in the bin - Attempting to restore it should fail or return not found

Unit Tests: `N/A`